

ASSESSMENT

CO2 - AT2

Name : Ch. Bhavya Sri

Reg no : 192411249

Course : CSA0612
Code

Course : Design and analysis
algorithms

Q 23) Selection Sort - Repeated Minimum elements

1. Problem Interpretation

The given dataset is:

$$A = [12, 5, 7, 5, 18, 9]$$

The objective is to sort the given array in ascending order using the selection Sort algorithm, which is a Brute force Sorting technique. The dataset contains duplicate minimum elements (5 and 5). We must observe how Selection Sort indicates repeated minimum values, display the array after every pass, Count the total of Comparisons and swaps, analyze the time Complexity, verify the final sorted result, and interpret the behavior of the algorithm.

Why Selection Sort?

- It is simple to understand and implement.
- It works Correctly even when duplicate values are present.
- It always performs the same number of Comparisons regardless of the input order.
- It requires only a small amount of extra memory (in-place sorting)

Selection Sort Algorithm

- Step 1: Start from the first element
Step 2: Find the minimum element in the unsorted portion.
Step 3: Swap the minimum element with the first unsorted position.
Step 4: Move to the next position.
Step 5: Repeat until the entire array becomes sorted.

Step-by-Step Sorting Process

Initial Array - $[12, 5, 7, 5, 18, 9]$

Pass 1 Current array: $[12, 5, 7, 5, 18, 9]$

Minimum element in the array = 5 (first Occurrence)

Swap 12 and 5

Result: $[5, 12, 7, 5, 18, 9]$

Comparisons = 5, Swaps = 1

Pass 2 Unsorted part: $[12, 7, 5, 18, 9]$

Minimum element = 5

Swap 12 and 5

Result: $[5, 5, 7, 12, 18, 9]$

Comparisons = 4, Swaps = 1

Pass 3

Unsorted part:

$[7, 12, 18, 9]$

Minimum element = 7,

— Already in correct position

Result: $[5, 5, 7, 12, 18, 9]$

Comparisons = 3, Swaps = 0

Pass 4 Unsorted part: $[12, 18, 9]$

Minimum element = 9

Swap 12 and 9

Result:

$[5, 5, 7, 9, 18, 12]$

Comparisons = 2, Swaps = 1

Pass 5

Unsorted part: $[18, 12]$

Minimum element = 12

Swap 18 and 12

Final array: $[5, 5, 7, 9, 12, 18]$

Comparisons = 1

Swaps = 1

Total Comparisons: $5 + 4 + 3 + 2 + 1 = 15$

Time Complexity Analysis

Best Case:

Even if the array is already sorted, of Selection Sort compares every element.

$$\text{Time Complexity} = O(n^2)$$

Average Case

Average Comparisons remains the same.

$$\text{Time Complexity} = O(n^2)$$

Worst Case:

When the array is in reverse order, Selection Sort still performs the same Comparisons.

$$\text{Time Complexity} = O(n^2)$$

$$\text{Space Complexity} = O(1)$$

Verification of the final Result

Final sorted array: $[5, 5, 7, 9, 12, 18]$

Checking the order:

$$5 \leq 5 \leq 7 \leq 9 \leq 12 \leq 18$$

Hence, the array is correctly stored in ascending order.